IT2080 - IP Project

Activity 2 - Requirements engineering

IT Number	Name	Email	Contact Number
IT23586352	Kasthuriarachchi GA	IT23586352@my.sliit.lk	071-6019649
IT23728226	Ashfan GM	IT23728226@my.sliit.lk	071-3896578
IT23647428	Pahasera AAV	IT23647428@my.sliit.lk	071-8850129
IT23603318	Shrihara LYP	IT23603318@my.sliit.lk	070-1599398
IT23668386	H.M.Imahari Pavaratha	IT23668386@my.sliit.lk	071-8486148

Stakeholders

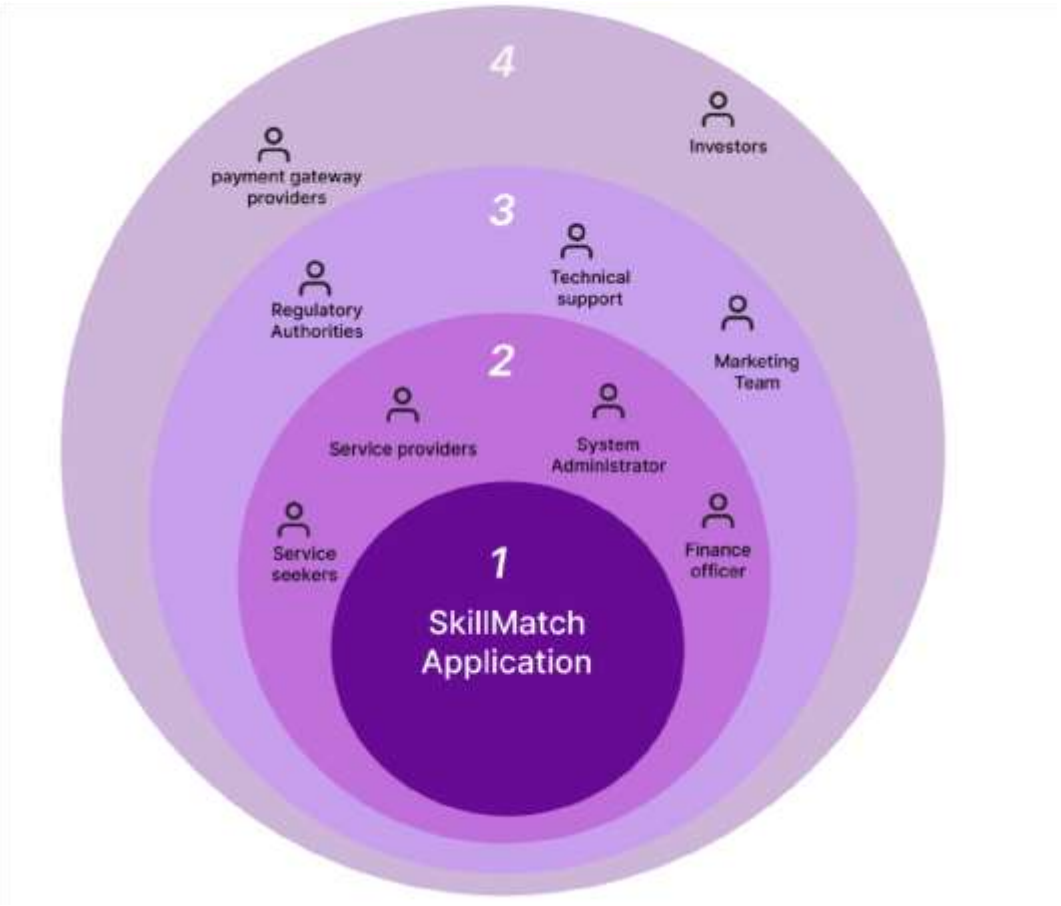
1. Service Seekers
2. Service Providers
3. System Admin
4. Finance Officer

External parties :

5. Marketing Team
6. Customer support team
7. Security Engineers

8. Legal Authorities

Onion Diagram



Functional Requirements

1. Client Functional Requirements

- Register and log in safely.
- Post service requests with details regarding location, date/time, budget, description.
- Get alerts when providers place bids.
- View and compare provider bids.
- Choose and book providers from bids or through direct booking.
- View booking history and current bookings.
- Leave ratings and reviews after service completion.
- Edit or cancel service requests before booking confirmation.
- Search for service providers by category or location.
- Communicate with providers via messaging or contact details if needed.

2. Service Provider Functional Requirements

- Register and create a detailed profile with services, pricing, availability.
- Get alerts for relevant service requests.
- Submit, edit, or withdraw bids on client requests.
- Manage and update service listings.
- Accept bookings and updating task statuses.
- View their work history and payments.
- Communicate with clients through the platform.
- View and respond to reviews from clients.

3. System Administrator Functional Requirements

- Manage user accounts (approve, block, reset passwords).
- Control service categories (add, edit, delete, approve new requests).
- Check and moderate content (service requests, bids, bookings, reviews).
- Handle complaints and problems.
- Access and review system reports and analytics.
- Send notifications and announcements to users.
- Watch for suspicious activity and keep the system secure.

4. Finance Officer Functional Requirements

- Track all payment transactions.

- Manage platform commission and service charges.
- Create financial reports (revenue summaries, payment history).
- Handle refunds and resolve payment issues.
- Collaborate with admin on financial and payment policies.
- Manage payment settings and keep them secure.

Non-Functional Requirements

Client	• **Security:** Safe login and data protection. • **Performance:** Fast browsing and booking. • **Availability:** Always accessible, 24/7. • **Usability:** Easy to use and navigate. • **Compatibility:** Works on all devices. • **Reliability:** Booking and payment must work without issues.
Service Provider	• **Security:** Secure access to job-related information. • **Performance:** Instant job alerts. • **Availability**: Always ready to manage jobs. • **Usability:** Simple dashboard for tasks. • **Compatibility:** Mobile and browser friendly. • **Reliability:** Accurate information on jobs and earnings.
System Administrator	• **Security:** Secure admin access. • **Performance:** Quick admin functions. • **Availability:** Access anytime to manage the system. • **Usability:** Easy tools for managing users and content. • **Maintainability:** Easy to update and fix. • **Auditability:** Tracks system activities.
Finance Officer	• **Security:** Safe handling of payment data. • **Performance:** Quick report and payment access. • **Availability:** Always able to view transactions. • **Usability:** Simple tools for finances. • **Auditability:** Keeps logs for transparency. • **Reliability:** Accurate financial records.

Technical requirements

Technical Requirements for the System (MERN Stack)
Backend & Database

Backend: Node.js and Express.js (routing and middleware).

Database: MongoDB

Data Model: MongoDB collections service seeker, service provider, system administrator, finance officer

API Handling: Postman.

Frontend

Frontend: React.js

CSS Framework: Normal CSS for easy and flexible styling

UI/UX Design: Responsive design to ensure compatibility of the user.

Authentication & Security

Authentication: JWT (JSON Web Tokens) and role-based access control (RBAC).

Authorization: Role-based access control (RBAC)

Password Security: Hashing all user passwords using bcrypt or argon2

Hosting & Deployment

Database Hosting: MongoDB Atlas

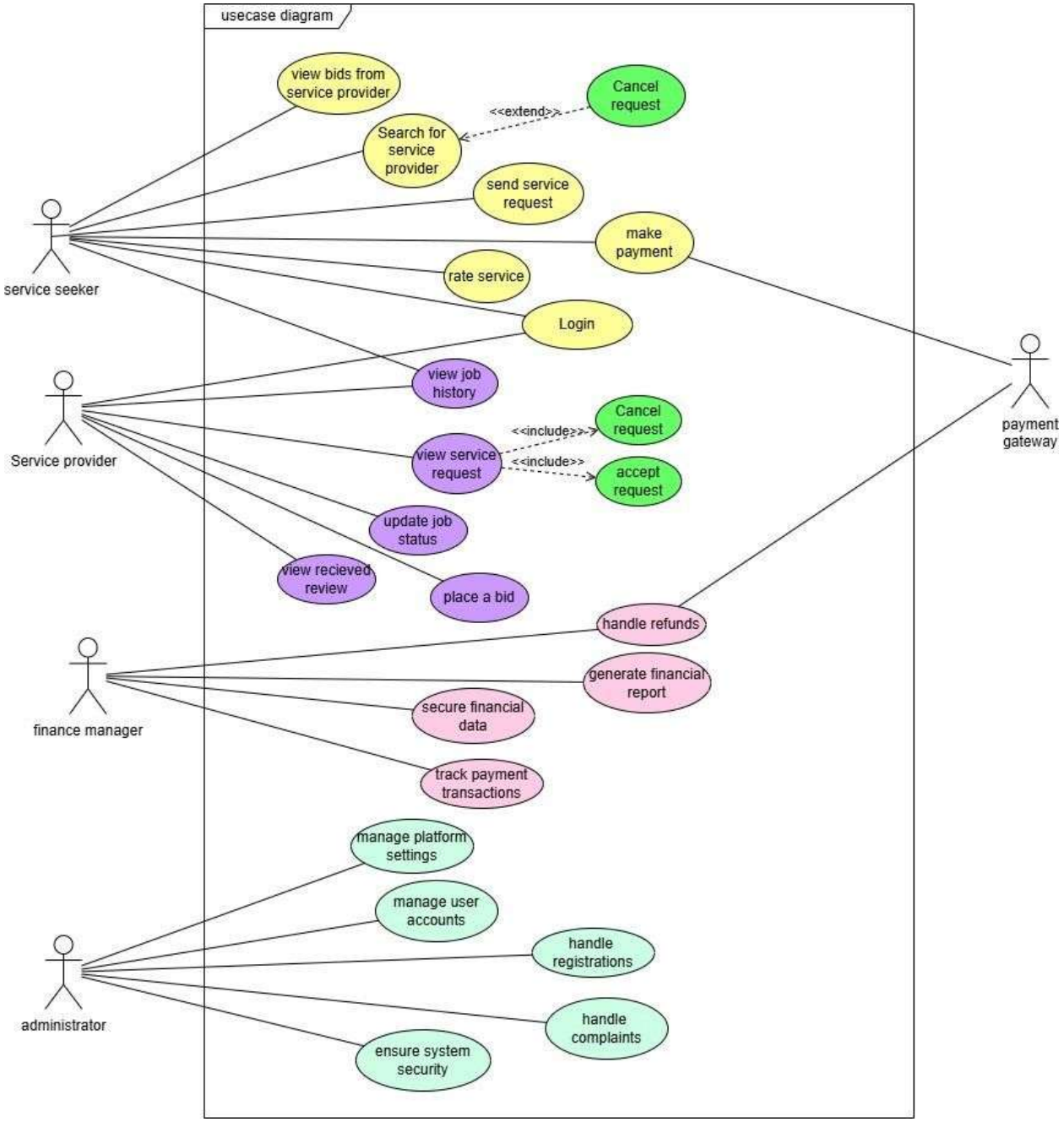
CI/CD Pipeline: Use GitHub Actions for continuous integration and deployment workflow.
Input Validation: Use express-validator to validate server-side request.

Performance

Database Indexing: Create indexes on frequently queried fields in MongoDB to improve data retrieval speed.

Frontend optimization.

Use case diagram



Use Cases

Usecase name:	Post a service request
Actor:	Service seeker

Goal:	Service seeker posts a service request for the service provider to respond
Overview:	Service seeker submits a service request form with proper details to the service provider.
Pre-conditions:	Service seeker must be logged in to the system.
Post-conditions:	Service requests are stored in the database.
Basic path/ alternative path	Basic: 1. Service seeker goes to the dashboard. 2. Service seeker selects to go to the " create appointment" page. 3. Service seeker fills out the request form. 4. Submit the request form. 5. The system validates inputs. 6. Save data to database. 7. Send message no notify service seeker about successful submission Alternative: • 3a.service seeker does not fill in all inputs in the request form,- the form will not submit and display an error message. • service seeker cancels before form submission- the record will not be created.
NFR and TR:	NFR: performance efficiency-react to user actions with in response time. Usability- usable to service seekers with various access. TR: frontend- react.js form with client side verification. Backend:express.js POST request handling Validations:express-validator

Usecase name:	Update job status
Actor:	Service provider
Goal:	Update the progress and status of the job.
Overview:	Update the status to show the current progress and notify the service seeker.
Pre-conditions:	Provider must be logged in to the system
Post-conditions:	• Job status must be updated. • Notify the service seeker of the current status.
Basic path/ alternative path	Basic path: 1. Logs in to the system. 2. Employee goes to the 'current jobs' page. 3. Employee approves an appointment. 4. Confirmation message will be sent to the customer. Alternative path: 3a.invalid job id -show error message 3b.invalid job status -show error message
NFR and TR:	Non-Functional Requirements: Availability- 24/7 access to the system. Usability- easy to change status by using a dropdown

Use case name:	Handle complains
Actor:	Administrator
Goal:	Review and solve complaints submitted by service seekers.
Overview:	Admin reads complaints in the complaints dashboard. Admin contacts the involved parties and solves the problems.
Pre-conditions:	A complaint must be submitted by a service seeker.
Post-conditions:	Complaint status is updated and generate reports. Administrator must send a solution to the problem.
Basic path/ alternative path	Basic path: 1. Administrator logs in to the system. 2. Access the complaint page. 3. Select one issue. 4. Review and clarify the ongoing issue with connected parties. 5. Update the complaint status. 6. Save the change and notify the service seeker. Alternative paths: 4a.administrator cannot connect a one or all parties. 5a.if complain is not reviewed mark it as pending admin review
NFR and TR:	NFR: Auditability- record complain logs. Security- restricted access. TR: Administrator ui-react.js dashboard

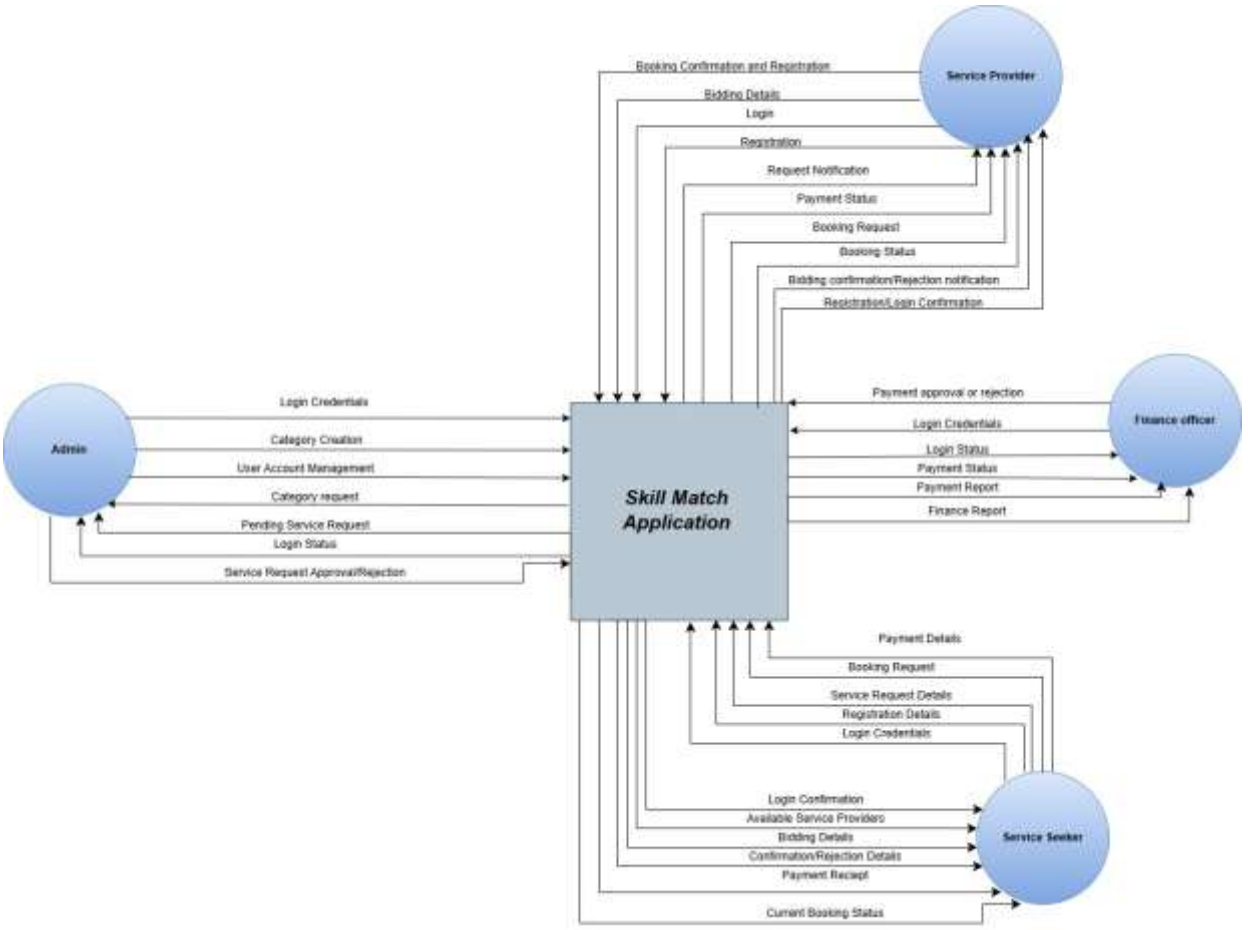
Usecase name:	Make payment
Actor:	Service seeker
Goal:	Allow the service seeker to pay for a completed job
Overview:	Service seeker sends a request and service provider completes the job. service seeker can proceed with payment.
Pre-conditions:	Service seeker must be logged in to the system Job status must be 'completed'.
Post-conditions:	• Payment proceeds successfully. • Store transaction details. • Notify the service seeker and service provider
Basic path/ ve path alternati	Basic path: 1. The service provider must mark the job as 'completed'. 2. Service seeker navigates to the payment page 3. The service seeker reviews the job and invoice. 4. Service seeker selects a payment method 5. Service seeker confirms and completes the payment 6. Notify all users with successful transection message. 7. Update order status to 'paid'. Alternative path: 3a: if payment details are invalid then, Display error message. 3b.Ask service seeker to retry again. 5a: unsuccessful payment due to transaction error - Retry the same payment method 5b.Choose another payment method 5c.Cancel transaction.

NFR and TR:	NFR- Security- secure client payments with encryption. Reliability- there will be no data loss during transactions. TR- Payment:integrate with help of payment gateway DB – mongo DB

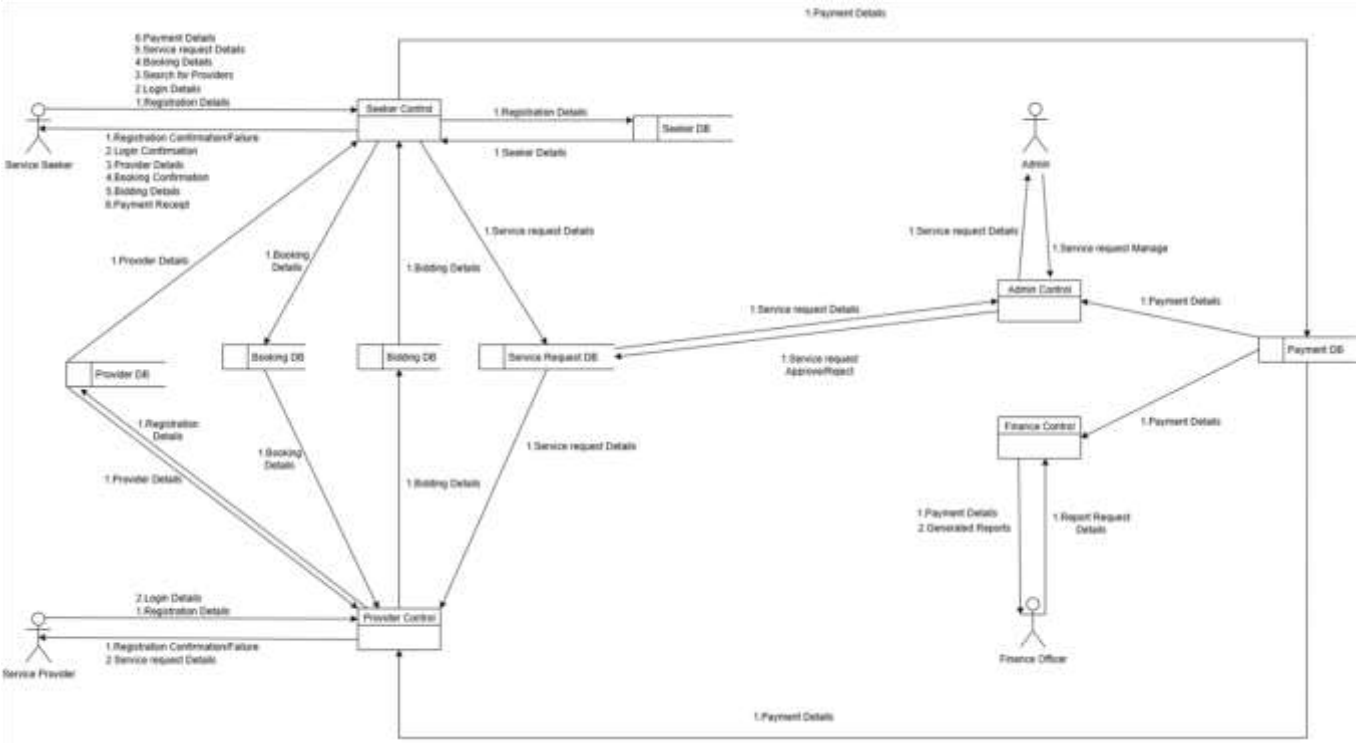
Use case name:	Handle refund
Actor:	Administrator, client
Goal:	
Overview:	Process which service seeker requests a refund due to dissatisfaction or job cancellation.
Pre-conditions:	Payment must be done . Refund application must be submitted with in refund period(7 days)
Post-conditions:	Review Refund request and approve or reject it.

Basic path/ alternative path	Basic path: 1. Service seeker goes to service history and selects thejob which want a refund. 2. Clicks on “request refund” and give a valid reasoning. 3. Refund request saved and notify the admin. 4. Admin logs in to system dashboard and view refund requests. 5. Admin decides to accept or decline requests. 6. System updates job status and notify the service seeker and service provider. Alternative paths: 2a.job is not able to refund. 6a.admin contact client for further clarification. 7a.refund fails and notify administrator for resolution.
NFR and TR:	NFR: All refund communications must be sent with in responsive time. All refund communications must be tracable. TR:payment gateway must support refund api calls

Context diagram - Level 0 DFD



Level 1 DFD diagram



Project Planning as a Team

The goal of SkillMatch's development is to create a digital link between Sri Lankan service providers and customers. By giving them a place to advertise their services and establish connections with potential customers in need of urgent or planned help, the platform is intended to assist people with a variety of skills, including tutors, electricians, cleaners, and more.

Our team started by identifying gaps in the current service hiring environment, where clients have trouble finding reliable providers quickly and local professionals are frequently underutilized. We collected functional and user-centered requirements through surveys, interviews, and case studies in order to inform the system's features.

To guarantee a modular and effective development process, each core module such as user account management, service listings, bidding, booking, payments, and review systems was distributed among team members.

Function of Members

Member	Function
H.M.Imahari Pavaratha	Service Request Module
Kasthuriarachchi GA	Bidding Module
Ashfan GM	Payment and Finance Module
Pahasera AAV	Booking Module
Shrihara LYP	Feedback and Review Module

Methodology

The project follows the Agile development methodology with regular feedback cycles and version control through GitHub to make collaboration and continuous improvement possible. We will develop in sprints (1-2 weeks) with iterative planning, coding, review, testing, and refactoring. Each sprint will deliver a working part of the system

- **Daily Stand-ups**

During our fast-paced daily stand-ups, each member reports their progress, shares any problems or challenges they encountered, and informs the team about their accomplishments of the previous day. The team becomes more focused and transparent as a result.

- **Sprint Planning**

At the start of each sprint, we collectively determine and prioritize essential features of the SkillMatch platform. We set sprint goals, task breakdown, and allocate tasks to team members based on module ownership and workload in this session.

- **Sprint Reviews**

We hold a review meeting at the conclusion of each sprint to present the work accomplished, illustrate the features implemented in SkillMatch, and gather feedback to ensure the deliverables are technically and user-friendly sufficiently

Communication & Collaboration – SkillMatch Project

- **Project Management**

For project management, we utilize GitHub Projects to schedule tasks, track progress, and allocate tasks efficiently within the team. This helps us stay in alignment with sprint goals and schedules while working under an Agile structure.

- **Version Control**

To keep our code base organized, avoid conflicts, and keep the development process running smoothly, we use Git with GitHub for versioning. Before merging into the main branch through pull requests, each team member is working on a specific feature branch and following a simple branching strategy.

- **Communication**

We officially utilize Microsoft Teams for chats and weekly meetings

- **Code Reviews**

Our development is based on modular CRUD functions that are shared among team members. Additionally, there are constant code reviews in which team members review each other's submissions for functionality, quality, and consistency. There is final approval and reviewing by the team leader before merging code into the main branch.